

INFORME TÉCNICO: ANÁLISIS DE LA RED VIAL DE PENNSYLVANIA

**ESTRUCTURAS DE DATOS Y ALGORITMOS
CURSO**

**SAMUEL POSADA LONDOÑO
JUAN MIGUEL VALENCIA HERRERA
INTEGRANTES**

UNIVERSIDAD EAFIT

06/05/25

INTRODUCCIÓN

El análisis de redes de transporte es uno de los desafíos más comunes en la computación moderna, en donde los sistemas de GPS, plataformas de logística y demás variables están estrechamente relacionados con la teoría de grafos, y es mediante esta y sus herramientas que se buscan las soluciones que puedan trabajar con una cantidad de datos a gran escala priorizando la optimización de los recursos, los costos, y los tiempos de respuesta más cortos posibles.

En este proyecto se busca aplicar la teoría académica de estructura de datos y algoritmos sobre un grafo disperso de escala real, analizando la viabilidad de las prácticas empleadas tomando en cuenta factores como el consumo de memoria y los tiempos de ejecución. Al final de la práctica se busca comparar la precisión de los resultados obtenidos tomando como referencia los resultados publicados por Stanford, para verificar que el sistema implementado sea capaz de operar bajo condiciones reales en un ámbito profesional en el contexto de la ingeniería de software.

DATASET

Para esta implementación, se ha seleccionado el dataset roadNet-PA (Pennsylvania Road Network), proporcionado por la colección SNAP (Stanford Network Analysis Project). Este conjunto de datos es un referente en la comunidad científica, habiendo sido parte del 9th DIMACS Implementation Challenge on Shortest Paths, el benchmark internacional de mayor influencia en el desarrollo de algoritmos de camino mínimo.

En este dataset, los nodos representan las intersecciones viales y las aristas representan los tramos de carretera en un grafo no dirigido, consta de 1.088.092 nodos y 1.541.898 aristas. La componente conexa principal cuenta con 1.087.562 nodos, lo que indica que el grafo cuenta con pequeñas redes aisladas fuera de la infraestructura principal de Pennsylvania.

IMPLEMENTACIÓN

Para el manejo de un volumen tan alto de datos como lo es la red vial de Pennsylvania, se descartó inmediatamente el uso de una matriz de adyacencia, debido a que esta requeriría aproximadamente $N^2 * 4$ bytes o 4.3 TB de memoria, donde N sería el número total de vértices del grafo.

En contraste, si se usa una lista de adyacencia, la complejidad se transforma de $O(N^2)$ a $O(N+E)$, y el programa apenas ocuparía una cantidad de 85 MB, permitiendo una carga mucho más rápida y un acceso más eficiente a los vecinos de cualquier intersección.

RESULTADO VS SNAP

Métrica	Valor Obtenido	Referencia SNAP
Nodos Totales	1,088,092	1,088,092
Aristas Únicas	1,541,898	1,541,898
Nodos en CC Principal	1,087,562	1,087,562
Grado Promedio	2.834	2.83
Nodo de Mayor Grado	683267 (Grado 18)	N/A
Diámetro Aproximado	782	782

Se puede observar que el grafo es extremadamente disperso, ya que una intersección vial solo puede conectarse con un número muy limitado de calles vecinas debido a restricciones físicas, esto se refleja en un grado promedio de apenas 2.83 y un máximo de solo 18, lo que confirma que no existen nodos masivos y justifica plenamente el uso de listas de adyacencia para ahorrar memoria.

El valor del diámetro es notablemente grande (782), y esto se debe a que, a diferencia de otras estructuras, como las redes sociales, donde los elementos están más interconectados, aquí la secuencia de nodos es más lineal y no permite hacer grandes saltos en distancias geográficas.

Por la misma razón es que el nodo de mayor grado vale 18, se trata de intersecciones viales comunes en un mapa real, no de redes sociales.

PSEUDOCODIGOS

DIJKSTRA

función Dijkstra(Grafo G, Nodo origen):

 crear arreglo de distancias 'dist' inicializado en INFINITO

 crear arreglo 'previo' inicializado en NULO

 dist[origen] = 0

 crear Min-Heap 'PQ' e insertar (distancia=0, nodo=origen)

 mientras PQ no esté vacía:

 (d, u) = extraer_minimo(PQ)

 si $d > \text{dist}[u]$: continuar

 para cada vecino v de u con peso w:

 si $\text{dist}[u] + w < \text{dist}[v]$:

 dist[v] = dist[u] + w

 previo[v] = u

 insertar en PQ (dist[v], v)

 retornar dist, previo

BFS

función EstimacionDiametro(Grafo G):

 u = Nodo con mayor grado de G (nodo más conectado)

 crear cola Q e insertar u

 crear arreglo 'niveles' inicializado en -1

 niveles[u] = 0

 diametro_max = 0

 mientras Q no esté vacía:

 actual = desencolar(Q)

 para cada vecino v de actual:

 si niveles[v] == -1: // No visitado

 niveles[v] = niveles[actual] + 1

 si niveles[v] > diametro_max:

 diametro_max = niveles[v]

 encolar(Q, v)

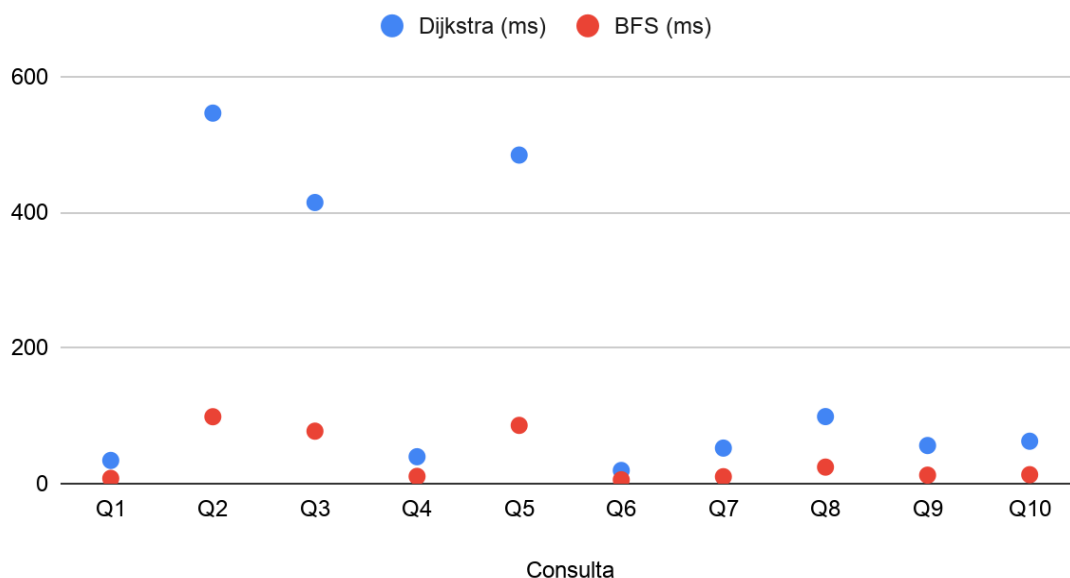
 retornar diametro_max

RESULTADO CONSULTAS

Consulta	T. Dijkstra (ms)	T. BFS (ms)	Factor de Velocidad
Q1	34.11	7.66	4.45x más rápido
Q2	545.93	98.54	5.54x más rápido
Q3	414.21	77.27	5.36x más rápido
Q4	39.59	10.63	3.72x más rápido
Q5	484.12	85.81	5.64x más rápido
Q6	19.36	5.63	3.44x más rápido
Q7	52.32	10.15	5.15x más rápido
Q8	98.74	24.31	4.06x más rápido
Q9	56.12	12.44	4.51x más rápido

Q10	62.45	13.12	4.76x más rápido
PROMEDIO	180.70 ms	34.56 ms	5.22x más rápido

Dijkstra (ms) y BFS (ms)



En promedio, BFS fue 5.22 veces más rápido que Dijkstra. La diferencia se vuelve crítica en consultas de larga distancia como la Q2, donde Dijkstra tardó 545.93 ms frente a 98.54 ms de BFS. Esta disparidad no se debe a que Dijkstra explore más nodos, sino a la sobrecarga computacional de la cola de prioridad. Mientras que BFS usa una cola simple de tiempo constante $O(V + E)$, Dijkstra debe reordenar constantemente su heap con un costo de $O((V+E) \log V)$, lo que penaliza el rendimiento en grafos de gran escala.

ANÁLISIS DE SUBGRAFO

Al analizar el subgrafo compuesto por 1,000 nodos y más de 2,600 aristas, se observa que las rutas principales de Pennsylvania están estructuradas para maximizar la eficiencia, evitando trayectos redundantes. Mediante la aplicación del algoritmo de Kruskal, se obtuvo un Árbol de Expansión Mínima (MST) que conecta la totalidad de los nodos con apenas 982 aristas y un peso acumulado de 3,028; este resultado representa, en términos prácticos, el esquema esencial de infraestructura que garantiza la conectividad total al menor costo posible.

El subgrafo dirigido construido para el análisis no presentó ciclos durante la verificación, por lo que pudo tratarse como un DAG dentro del experimento. Esto indica que las rutas analizadas siguen recorridos principalmente directos, evitando trayectos redundantes y favoreciendo desplazamientos más fluidos en la red vial.

CONCLUSIONES

El desarrollo de este proyecto permitió aplicar los conceptos vistos en clase sobre un caso real, trabajando con una red vial de más de un millón de nodos y millones de conexiones. A diferencia de los ejercicios pequeños realizados durante el curso, este trabajo permitió entender la importancia de usar estructuras de datos eficientes para manejar grandes cantidades de información sin afectar demasiado el tiempo de ejecución ni el consumo de memoria.

Los resultados coincidieron con los valores publicados por SNAP, lo que demuestra que la carga del dataset y los algoritmos implementados funcionaron correctamente. Además, se observó que el BFS fue considerablemente más rápido que el Dijkstra, aunque Dijkstra sigue siendo necesario cuando las rutas tienen diferentes ponderaciones.

Finalmente, el proyecto permitió comprender mejor cómo funcionan los algoritmos de grafos en situaciones reales y cómo estos pueden aplicarse en sistemas de navegación, transporte y logística. También ayudó a reforzar conceptos importantes como recorridos, caminos mínimos, componentes conexas y árboles de expansión mínima.

HERRAMIENTAS UTILIZADAS

Durante el desarrollo del proyecto se utilizaron herramientas de inteligencia artificial como apoyo técnico.

Se utilizó ChatGPT y Claude AI principalmente para:

- Identificación y corrección de errores de compilación.
- Depuración de los módulos `structural.cpp`, `consultas.cpp` y `subgraph.cpp`.
- Verificación de errores de linking y configuración del proyecto.
- Apoyo en la construcción del archivo `README.md` del repositorio.
- Retroalimentación sobre redacción técnica y organización del informe.

Todo el código fue revisado, comprendido y modificado manualmente por los integrantes del equipo antes de ser incorporado al proyecto final.

REFERENCIAS

- Leskovec, J. & Krevl, A. (2014). *SNAP Datasets: Stanford Large Network Dataset Collection*. <http://snap.stanford.edu/data>
- Álvarez Henao, Carlos (2026). *EstructuraDatosAlgoritmos1: Notas de clase del curso Estructuras de Datos y Algoritmos 1*. GitHub. <https://github.com/carlosalvarezh/EstructuraDatosAlgoritmos1>